

K L Deemed to be University

Department of Computer Science and Engineering | 2026-2027, Odd Semester
Database Systems Engineering and Distributed Backend Development (25CS1302E)

Practical Experiment 5: FastAPI Backend and Security Suite

Name	Habeeb	Roll No.	2520090054
Course	25CS1302E	CO Mapping	CO3
Session	Practical 5	Date	

Aim

To build a FastAPI 'Gatekeeper' for BookFlow with a Pydantic BookSchema (13-character ISBN, positive price, no future published_date), an async POST /books route that saves to PostgreSQL with SQLAlchemy 2.x async and logs a background task, an async pgvector search endpoint using <=>, and a member registration route that hashes passwords with passlib bcrypt, protected by a reusable dependency-injected security guard.

Program

main.py

```
"""
Experiment 5: FastAPI Backend and Security Suite for BookFlow.

Run:
    pip install fastapi uvicorn "sqlalchemy[asyncio]" asyncpg pgvector \
        "passlib[bcrypt]" bcrypt==4.0.1 pyjwt
    uvicorn main:app --reload --port 8000
"""
import logging
import os
import re
from contextlib import asynccontextmanager
from datetime import date, datetime, timedelta, timezone
from decimal import Decimal
from typing import AsyncIterator, List, Optional

import jwt
from fastapi import BackgroundTasks, Depends, FastAPI, HTTPException, Query, status
from fastapi.security import OAuth2PasswordBearer, OAuth2PasswordRequestForm
from passlib.context import CryptContext
from pgvector.sqlalchemy import Vector
from pydantic import BaseModel, ConfigDict, EmailStr, Field, field_validator
from sqlalchemy import Date, Integer, Numeric, String, select, text
from sqlalchemy.exc import IntegrityError
from sqlalchemy.ext.asyncio import AsyncSession, async_sessionmaker, create_async_engine
from sqlalchemy.orm import DeclarativeBase, Mapped, mapped_column

# -----
# Configuration
# -----
DATABASE_URL = os.getenv(
    "DATABASE_URL", "postgresql+asyncpg://postgres:postgres@localhost:5432/bookflow_db"
)
JWT_SECRET = os.getenv("JWT_SECRET", "change-this-secret-in-production")
JWT_ALGORITHM = "HS256"
TOKEN_MINUTES = 30
EVENT_LOG_FILE = os.getenv("EVENT_LOG_FILE", "book_events.log")

logging.basicConfig(level=logging.INFO, format="%(asctime)s %(levelname)s %(message)s")
logger = logging.getLogger("bookflow")

engine = create_async_engine(DATABASE_URL, pool_size=10, max_overflow=20)
SessionLocal = async_sessionmaker(engine, expire_on_commit=False)
pwd_context = CryptContext(schemes=["bcrypt"], deprecated="auto")
oauth2_scheme = OAuth2PasswordBearer(tokenUrl="/auth/login")

# -----
# ORM models (SQLAlchemy 2.x declarative mapping)
# -----
```

```

class Base(DeclarativeBase):
    pass

class Book(Base):
    __tablename__ = "books"

    book_id: Mapped[int] = mapped_column(Integer, primary_key=True)
    title: Mapped[str] = mapped_column(String(200), nullable=False)
    isbn: Mapped[str] = mapped_column(String(13), unique=True, nullable=False)
    published_year: Mapped[Optional[int]] = mapped_column(Integer)
    published_date: Mapped[Optional[date]] = mapped_column(Date)
    price: Mapped[Optional[Decimal]] = mapped_column(Numeric(10, 2))
    embedding: Mapped[Optional[List[float]]] = mapped_column(Vector(3))

class Member(Base):
    __tablename__ = "members"

    member_id: Mapped[int] = mapped_column(Integer, primary_key=True)
    full_name: Mapped[str] = mapped_column(String(100), nullable=False)
    email: Mapped[str] = mapped_column(String(150), unique=True, nullable=False)
    password_hash: Mapped[Optional[str]] = mapped_column(String(255))

# -----
# Pydantic schemas
# -----
class BookSchema(BaseModel):
    title: str = Field(min_length=1, max_length=200)
    isbn: str = Field(min_length=13, max_length=13, description="Exactly 13 characters")
    price: Decimal = Field(gt=0, max_digits=10, decimal_places=2)
    published_date: date

    @field_validator("isbn")
    @classmethod
    def isbn_must_be_digits(cls, value: str) -> str:
        if not value.isdigit():
            raise ValueError("isbn must contain only digits")
        return value

    @field_validator("published_date")
    @classmethod
    def date_not_in_future(cls, value: date) -> date:
        if value > date.today():
            raise ValueError("published_date cannot be in the future")
        return value

class BookOut(BaseModel):
    model_config = ConfigDict(from_attributes=True)

    book_id: int
    title: str
    isbn: str
    price: Optional[Decimal] = None
    published_date: Optional[date] = None

class SearchHit(BaseModel):
    book_id: int
    title: str
    cosine_distance: float

class MemberRegister(BaseModel):
    full_name: str = Field(min_length=2, max_length=100)
    email: EmailStr
    password: str = Field(min_length=8, max_length=72)

class MemberOut(BaseModel):
    model_config = ConfigDict(from_attributes=True)

    member_id: int
    full_name: str
    email: str

```

```

class Token(BaseModel):
    access_token: str
    token_type: str = "bearer"

# -----
# Embedding helper
# -----
# Three theme dimensions, matching the VECTOR(3) mock embeddings of Experiment 4:
# [space / science fiction, computer science, self-improvement]
# In production this function is replaced by a real model such as
# sentence-transformers "all-MiniLM-L6-v2" (384 dimensions).
THEME_KEYWORDS = (
    {"space", "planet", "mars", "alien", "astronaut", "galaxy", "star", "desert", "sci-fi"},
    {"code", "software", "database", "algorithm", "data", "programming", "sql", "system"},
    {"habit", "habits", "focus", "productivity", "life", "mind", "success", "work"},
)

def embed_text(query: str) -> List[float]:
    """Convert free text into a normalised 3-dimensional theme vector."""
    words = re.findall(r"[a-z\-\-]", query.lower())
    scores = [0.1 + sum(word in keywords for word in words) for keywords in THEME_KEYWORDS]
    total = sum(scores)
    return [round(score / total, 4) for score in scores]

# -----
# Dependencies (dependency injection)
# -----
async def get_db() -> AsyncIterator[AsyncSession]:
    """Provide one async database session per request."""
    async with SessionLocal() as session:
        yield session

async def get_current_member(
    token: str = Depends(oauth2_scheme), db: AsyncSession = Depends(get_db)
) -> Member:
    """Reusable security guard: allows the request only for a logged-in member."""
    unauthorized = HTTPException(
        status_code=status.HTTP_401_UNAUTHORIZED,
        detail="Invalid or expired token",
        headers={"WWW-Authenticate": "Bearer"},
    )
    try:
        payload = jwt.decode(token, JWT_SECRET, algorithms=[JWT_ALGORITHM])
        member_id = int(payload["sub"])
    except (jwt.PyJWTError, KeyError, ValueError):
        raise unauthorized
    member = await db.get(Member, member_id)
    if member is None:
        raise unauthorized
    return member

# -----
# Background task
# -----
def log_new_book_event(book_id: int, title: str, member_email: str) -> None:
    """Runs after the HTTP response has been sent to the client."""
    timestamp = datetime.now(timezone.utc).isoformat(timespec="seconds")
    with open(EVENT_LOG_FILE, "a", encoding="utf-8") as handle:
        handle.write(f"{timestamp} | New Book Added | id={book_id} | "
                    f"title={title} | by={member_email}\n")

# -----
# Application
# -----
@asynccontextmanager
async def lifespan(_: FastAPI) -> AsyncIterator[None]:
    """Startup: add the columns this experiment needs. Shutdown: close the pool."""
    async with engine.begin() as conn:
        await conn.execute(text("CREATE EXTENSION IF NOT EXISTS vector"))
        await conn.execute(text("ALTER TABLE books ADD COLUMN IF NOT EXISTS price NUMERIC(10,2)"))
        await conn.execute(text("ALTER TABLE books ADD COLUMN IF NOT EXISTS published_date DATE"))

```

```

        await conn.execute(text("ALTER TABLE books ADD COLUMN IF NOT EXISTS embedding VECTOR(3)"))
        await conn.execute(
            text("ALTER TABLE members ADD COLUMN IF NOT EXISTS password_hash VARCHAR(255)")
        )
    logger.info("Database schema ready")
    yield
    await engine.dispose()

app = FastAPI(title="BookFlow API", version="1.0.0", lifespan=lifespan)

@app.post("/auth/register", response_model=MemberOut, status_code=status.HTTP_201_CREATED)
async def register(payload: MemberRegister, db: AsyncSession = Depends(get_db)) -> Member:
    member = Member(
        full_name=payload.full_name,
        email=payload.email.lower(),
        password_hash=pwd_context.hash(payload.password),
    )
    db.add(member)
    try:
        await db.commit()
    except IntegrityError:
        await db.rollback()
        raise HTTPException(status.HTTP_409_CONFLICT, "Email is already registered")
    await db.refresh(member)
    return member

@app.post("/auth/login", response_model=Token)
async def login(
    form: OAuth2PasswordRequestForm = Depends(), db: AsyncSession = Depends(get_db)
) -> Token:
    result = await db.execute(select(Member).where(Member.email == form.username.lower()))
    member = result.scalar_one_or_none()
    if member is None or not member.password_hash or not pwd_context.verify(
        form.password, member.password_hash
    ):
        raise HTTPException(status.HTTP_401_UNAUTHORIZED, "Incorrect email or password")
    expires = datetime.now(timezone.utc) + timedelta(minutes=TOKEN_MINUTES)
    token = jwt.encode({"sub": str(member.member_id), "exp": expires}, JWT_SECRET, JWT_ALGORITHM)
    return Token(access_token=token)

@app.post("/books", response_model=BookOut, status_code=status.HTTP_201_CREATED)
async def create_book(
    payload: BookSchema,
    background_tasks: BackgroundTasks,
    db: AsyncSession = Depends(get_db),
    member: Member = Depends(get_current_member),
) -> Book:
    book = Book(
        title=payload.title,
        isbn=payload.isbn,
        price=payload.price,
        published_date=payload.published_date,
        published_year=payload.published_date.year,
        embedding=embed_text(payload.title),
    )
    db.add(book)
    try:
        await db.commit()
    except IntegrityError:
        await db.rollback()
        raise HTTPException(status.HTTP_409_CONFLICT, "A book with this ISBN already exists")
    await db.refresh(book)
    background_tasks.add_task(log_new_book_event, book.book_id, book.title, member.email)
    return book

@app.get("/books/search", response_model=List[SearchHit])
async def search_books(
    q: str = Query(min_length=2, max_length=200),
    limit: int = Query(default=3, ge=1, le=50),
    db: AsyncSession = Depends(get_db),
) -> List[SearchHit]:
    query_vector = embed_text(q)
    distance = Book.embedding.cosine_distance(query_vector) # SQL operator <=>

```

```

statement = (
    select(Book.book_id, Book.title, distance.label("cosine_distance"))
    .where(Book.embedding.is_not(None))
    .order_by(distance)
    .limit(limit)
)
rows = (await db.execute(statement)).all()
return [
    SearchHit(book_id=r.book_id, title=r.title, cosine_distance=round(r.cosine_distance, 4))
    for r in rows
]

```

test_p5.sh (API test calls)

```

#!/bin/bash
# Experiment 5 - API test script (run after: uvicorn main:app --port 8000)
BASE=http://127.0.0.1:8000
call() { # prints the request, the HTTP status, and the pretty-printed JSON body
    echo "\$ curl \$*"
    body=$(curl -s -w '\n%{http_code}' "$@" )
    code=$(echo "$body" | tail -n1); json=$(echo "$body" | sed '$d')
    echo "HTTP $code"
    [ -n "$json" ] && echo "$json" | python3 -m json.tool --indent 2
    echo
}

echo "### 1. Register a member (password stored as a bcrypt hash)"
call -X POST $BASE/auth/register -H 'Content-Type: application/json' \
    -d '{"full_name":"Habeeb","email":"habeeb.api@kluniversity.in","password":"Secure@2026"}'

echo "### 2. Login and receive a JWT access token"
TOKEN=$(curl -s -X POST $BASE/auth/login -d 'username=habeeb.api@kluniversity.in&password=Secure@2026' \
    | python3 -c 'import sys,json;print(json.load(sys.stdin)["access_token"])')
echo "\$ curl -X POST $BASE/auth/login -d 'username=habeeb.api@kluniversity.in&password=*****'"
echo "HTTP 200 -> access_token: ${TOKEN:0:36}... (token_type: bearer)"
echo

echo "### 3. Wrong password is rejected"
call -X POST $BASE/auth/login -d 'username=habeeb.api@kluniversity.in&password=wrongpass'

echo "### 4. POST /books without a token is rejected by the security guard"
call -X POST $BASE/books -H 'Content-Type: application/json' \
    -d '{"title":"Hitchhiker Guide to the Galaxy","isbn":"9780345391803","price":399.00,"published_date":"1995-09-27"}'

echo "### 5. POST /books with a valid token (async insert + background task)"
call -X POST $BASE/books -H "Authorization: Bearer $TOKEN" -H 'Content-Type: application/json' \
    -d '{"title":"Hitchhiker Guide to the Galaxy","isbn":"9780345391803","price":399.00,"published_date":"1995-09-27"}'

echo "### 6. Validation: ISBN not 13 characters, negative price, future date"
call -X POST $BASE/books -H "Authorization: Bearer $TOKEN" -H 'Content-Type: application/json' \
    -d '{"title":"Bad Book","isbn":"12345","price":-50,"published_date":"2030-01-01"}'

echo "### 7. Semantic search: 'stories about space and planets'"
call -G $BASE/books/search --data-urlencode 'q=stories about space and planets' -d limit=3

echo "### 8. Semantic search: 'learn database and software code'"
call -G $BASE/books/search --data-urlencode 'q=learn database and software code' -d limit=3

echo "### 9. Background task output (book_events.log)"
sleep 1
echo "\$ cat book_events.log"
cat book_events.log
echo

echo "### 10. Stored password hash (never the plain password)"
echo "\$ psql -c \"SELECT email, LEFT(password_hash, 29) || '...' AS password_hash FROM members WHERE password_hash IS NOT NULL;\""
psql -h /tmp/pg -U postgres -d bookflow_db -X -c "SELECT email, LEFT(password_hash, 29) || '...' AS password_hash FROM members WHERE password_hash IS NOT NULL;"

```

Output

uvicorn main:app --port 8000

```

INFO:      Started server process [1192]
INFO:      Waiting for application startup.
2026-10-02 12:07:40,563 INFO Database schema ready
INFO:      Application startup complete.
INFO:      Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)

```

bash test_p5.sh

```

### 1. Register a member (password stored as a bcrypt hash)
$ curl -X POST http://127.0.0.1:8000/auth/register -H Content-Type: application/json -d
{"full_name":"Habeeb","email":"habeeb.api@kluniversity.in","password":"Secure@2026"}
HTTP 201
{
  "member_id": 9,
  "full_name": "Habeeb",
  "email": "habeeb.api@kluniversity.in"
}

### 2. Login and receive a JWT access token
$ curl -X POST http://127.0.0.1:8000/auth/login -d 'username=habeeb.api@kluniversity.in&password=*****'
HTTP 200 -> access_token: eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9... (token_type: bearer)

### 3. Wrong password is rejected
$ curl -X POST http://127.0.0.1:8000/auth/login -d username=habeeb.api@kluniversity.in&password=wrongpass
HTTP 401
{
  "detail": "Incorrect email or password"
}

### 4. POST /books without a token is rejected by the security guard
$ curl -X POST http://127.0.0.1:8000/books -H Content-Type: application/json -d {"title":"Hitchhiker Guide to
the Galaxy","isbn":"9780345391803","price":399.00,"published_date":"1995-09-27"}
HTTP 401
{
  "detail": "Not authenticated"
}

### 5. POST /books with a valid token (async insert + background task)
$ curl -X POST http://127.0.0.1:8000/books -H Authorization: Bearer $TOKEN -H Content-Type: application/json -d
{"title":"Hitchhiker Guide to the Galaxy","isbn":"9780345391803","price":399.00,"published_date":"1995-09-27"}
}
HTTP 201
{
  "book_id": 15,
  "title": "Hitchhiker Guide to the Galaxy",
  "isbn": "9780345391803",
  "price": "399.00",
  "published_date": "1995-09-27"
}

### 6. Validation: ISBN not 13 characters, negative price, future date
$ curl -X POST http://127.0.0.1:8000/books -H Authorization: Bearer $TOKEN -H Content-Type: application/json -d
{"title":"Bad Book","isbn":"12345","price":-50,"published_date":"2030-01-01"}
HTTP 422
{
  "detail": [
    {
      "type": "string_too_short",
      "loc": [
        "body",
        "isbn"
      ],
      "msg": "String should have at least 13 characters",
      "input": "12345",
      "ctx": {
        "min_length": 13
      }
    },
    {
      "type": "greater_than",
      "loc": [
        "body",
        "price"
      ],
      "msg": "Input should be greater than 0",
      "input": -50,
      "ctx": {
        "gt": 0
      }
    },
    {
      "type": "value_error",
      "loc": [
        "body",

```

```

        "published_date"
    ],
    "msg": "Value error, published_date cannot be in the future",
    "input": "2030-01-01",
    "ctx": {
        "error": {}
    }
}
]
}

### 7. Semantic search: 'stories about space and planets'
$ curl -G http://127.0.0.1:8000/books/search --data-urlencode q=stories about space and planets -d limit=3
HTTP 200
[
  {
    "book_id": 15,
    "title": "Hitchhiker Guide to the Galaxy",
    "cosine_distance": 0.0
  },
  {
    "book_id": 3,
    "title": "The Martian",
    "cosine_distance": 0.0008
  },
  {
    "book_id": 7,
    "title": "Dune",
    "cosine_distance": 0.0009
  }
]

### 8. Semantic search: 'learn database and software code'
$ curl -G http://127.0.0.1:8000/books/search --data-urlencode q=learn database and software code -d limit=3
HTTP 200
[
  {
    "book_id": 9,
    "title": "Designing Data-Intensive Applications",
    "cosine_distance": 0.0001
  },
  {
    "book_id": 13,
    "title": "Introduction to Algorithms",
    "cosine_distance": 0.0001
  },
  {
    "book_id": 2,
    "title": "Database System Concepts",
    "cosine_distance": 0.0003
  }
]

### 9. Background task output (book_events.log)
$ cat book_events.log
2026-10-02T12:08:03+00:00 | New Book Added | id=15 | title=Hitchhiker Guide to the Galaxy |
by=habeeb.api@kluniversity.in

### 10. Stored password hash (never the plain password)
$ psql -c "SELECT email, LEFT(password_hash, 29) || '...' AS password_hash FROM members WHERE password_hash IS
NOT NULL;"
      email      | password_hash
-----+-----
 habeeb.api@kluniversity.in | $2b$12$b8H65EemK35WHSI2JTZjc....
(1 row)

```

Result

Registration stored only a bcrypt hash (\$2b\$12\$...), login issued a JWT, and the `get_current_member` dependency rejected the unauthenticated POST with 401. The authenticated POST `/books` returned 201 and the background task wrote the 'New Book Added' line after the response. Pydantic returned 422 with three errors for a 5-character ISBN, a negative price and a future date. The async pgvector search ranked space books for the space query and database books for the software query.